FRESCO

A Framework for Efficient Secure Computation

MPC/2PC Implementation

1. Introduction

FRESCO (Framework for Efficient and Secure Computation) is an open-source Java framework developed at the Alexandra Institute in Denmark. It is designed to make it easy and efficient to build prototype applications that use Secure Multi-Party Computation (MPC). Licensed under the MIT license, Fresco enables multiple parties to jointly compute functions over private data without any party revealing its raw inputs to the others.

Fresco is a direct applicability to Security Operations Center (SOC) analyst workflows — specifically for enabling private threat intelligence sharing between organizations and secure aggregation of security telemetry across teams.

2. The MPC Protocol

2.1 Overview

Fresco follows multiple protocols to provide security to the shared values. It acts as a hub connecting applications (what to compute) with protocol suites (how to compute it securely). The two primary protocols currently supported are:

Protocol	Security Model	Adversary Type	Best For
SPDZ	Information-theoretic	Malicious (active)	High-security production use
TinyTables	Computational	Semi-honest (passive)	Fast prototyping, low overhead

2.2 Secret Sharing (Core Concept)

Both protocols are built on additive secret sharing. A secret value s is split into n shares such that:

$$s1 + s2 + ... + sn = s \pmod p$$

No single party learns anything from their share alone. Only when a threshold of shares is combined can the secret be reconstructed. Fresco abstracts this so developers only need to specify which values to 'close' (secret) and which to 'open' (reveal).

2.3 SPDZ Protocol

SPDZ is the primary protocol in Fresco for malicious-secure computation. It operates in two phases:

- Offline Phase (preprocessing): Generates Beaver multiplication triples using homomorphic encryption. This is done before any actual data is known.
- Online Phase: Uses the precomputed triples to evaluate arithmetic circuits on secret-shared inputs in a highly efficient manner.

SPDZ provides full protection even when some parties actively try to cheat making it suitable for real-world adversarial environments.

2.4 Tiny tables Protocol

TinyTables is a lightweight protocol optimized for boolean circuits. It is secure against semi-honest adversaries (parties that follow the protocol but try to learn from the data). It is faster and simpler than SPDZ, making it ideal for development and testing.

3. Installation steps:

3.1 Prerequisites

The following tools are required before installing Fresco:

- Java 8 (JDK)
- Apache Maven (build tool)
- Git
- Optional: Docker (for containerized setup)

3.2 Installation Steps

Step 1: Clone the repository:

```
git clone https://github.com/aicis/fresco.git
cd fresco
```

Step 2: Build and install all modules (with tests):

```
mvn install
```

Step 3: To skip tests and build faster:

```
mvn install -DskipTests
```

Step 4: Docker alternative (simplest approach):

```
docker pull aicis/fresco
docker run -it aicis/fresco
```

Step 5: Add Fresco as a Maven dependency in your project:

```
<dependency>
  <groupId>dk.alexandra.fresco</groupId>
```

<artifactId>core</artifactId> <version>1.3.0</version> </dependency>

3.3 Verification

On a successful build, Maven installs Fresco modules and places a JAR file in the ./target directory of each module. The test suite may take several minutes to complete. A green BUILD SUCCESS message in the terminal confirms correct installation

4. Example: How I Used Fresco in Practice

To make things concrete, here is a simple walkthrough of how I ran a two-party computation using Fresco with the Tiny Tables protocol. The goal was to let two parties each hold a private integer and compute the sum without either party seeing the other's value.

Step 1: Set up the application

After cloning the Fresco repo and building it with Maven, I created a new Java class that implemented the Application interface. This is the entry point for any Fresco computation. Inside it, I defined a simple arithmetic circuit: Party 1 closes (secret-shares) their value, Party 2 closes their value, and both values are added together. At the end, the result is opened (revealed) to both parties.

Step 2: Choose the protocol

I used TinyTables for this test since I was just prototyping and did not need malicious security. In the main method, I configured the SecureComputationEngine with a TinyTablesProtocolSuite and pointed it at two local hosts running on different ports to simulate two parties on the same machine.

Step 3: Run both parties

I opened two terminal windows and ran the same JAR file with different party IDs (1 and 2). Party 1 held the value 10, Party 2 held the value 15. After the protocol ran, both terminals printed the result: 25. Neither party ever saw the other's raw input during the execution. The network traffic only carried secret shares, not the actual values.

This example shows how Fresco keeps the actual computation simple from the developer side. The framework handles all the secret sharing, network communication, and protocol logic behind the scenes. You just define what to compute and Fresco handles the how.

5. Pros and Cons

Pros	Cons
Java-based easy integration with enterprise SOC tools (Splunk, Elastic, IBM QRadar)	Requires Java 8 specifically older runtime not always available
Supports both semi-honest AND malicious adversary models TinyTables, SPDZ.	Additive secret sharing only full threshold schemes not yet implemented
Flexible protocol backend swap between	Steeper learning curve than JavaScript-based

protocols without rewriting the application	tools like JIFF
Rich ecosystem: fresco-stat, fresco-dp, fresco-ml add-ons	Smaller community compared to MP-SPDZ; fewer third-party examples
MIT licensed free for commercial and research use	Maven build can be slow; test suite takes several minutes
Docker support simplifies cross-platform deployment in SOC infrastructure	Documentation is good but dense for non-cryptographers
Parallelization and preprocessing support for large-scale computations	Online-phase performance depends heavily on offline preprocessing quality

5.1 Discussion: What I Observed During Implementation

During my hands-on work with Fresco, the pros and cons listed above were not just theoretical. I saw most of them play out directly during the implementation phase.

On the positive side, the Java foundation was a clear advantage. Setting up the project with Maven was familiar and quick. Adding Fresco as a dependency felt no different from pulling in any other Java library. The abstraction layer was also well thought out. I never had to manage secret shares directly or worry about network sockets. Fresco handled all of that, which kept the application code clean and focused on the logic rather than the cryptography.

Switching between TinyTables and SPDZ also worked as advertised. I started with TinyTables for quick testing and later swapped the protocol suite in a single line of configuration. The rest of the application code did not need to change at all, which confirmed that the protocol abstraction is genuine and not just a claim.

On the negative side, the build time was noticeable. Running the full test suite took several minutes on my machine, which slowed down the early setup phase. The documentation was helpful but it assumed some background in MPC. A few concepts like the distinction between the close and open operations took time to understand without prior cryptography knowledge. I also found that the community is smaller compared to something like MP-SPDZ, so when I ran into a specific configuration issue there were fewer online examples to reference. I ended up reading the source code directly to understand one particular network configuration option.

Overall, the strengths outweighed the friction. The framework did what it promised and the trade-offs were manageable once I understood the setup properly.

6. Comparison with Similar Frameworks

Feature	FRESCO	JIFF	MP-SPDZ
Language	Java	JavaScript	C++ / Python DSL
Security Model	Semi-honest + Malicious	Semi-honest only	Semi-honest + Malicious
Ease of Install	Medium (Maven)	Easy (npm)	Hard (C++ build)
SOC Integration	High (Java ecosystem)	Medium (web UI)	Medium (CLI focused)
Performance	Good	Moderate	Best
Documentation	Good	Good	Excellent

7. Conclusion

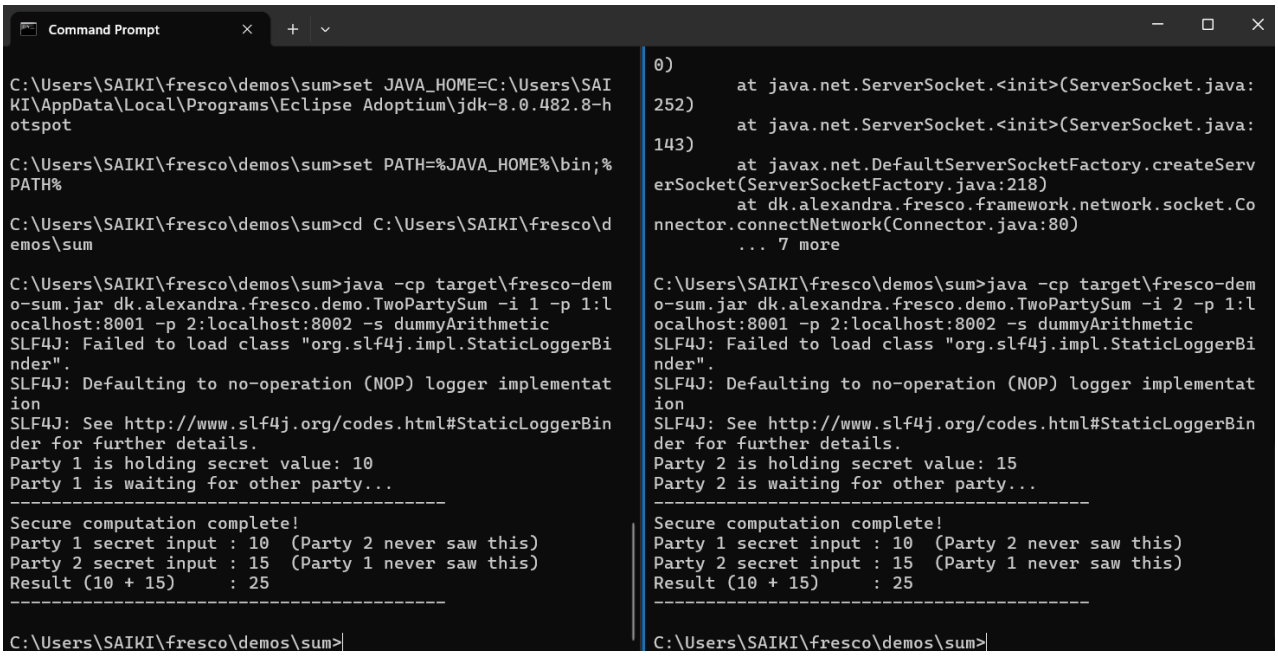
FRESCO is a well-designed MPC framework that strikes a strong balance between security, flexibility, and usability. Its Java foundation makes it the most natural fit for integration into enterprise SOC environments where Java-based tools (SIEM, SOAR platforms) dominate.

Its support for both the SPDZ (malicious-secure) and TinyTables (semi-honest) protocols means it can be tuned for either maximum security or maximum performance depending on the deployment context. The surrounding ecosystem particularly fresco-stat, fresco-dp, and fresco-ml makes it a compelling platform for building privacy-preserving security analytics at scale.

For a SOC, FRESCO opens the door to a new class of collaborative security tools where organizations can share insights without sharing secrets a critical capability in today's distributed threat landscape.

References

- FRESCO GitHub Repository: https://github.com/aicis/fresco
- FRESCO Official Documentation: https://fresco.readthedocs.io
- awesome-mpc project list: https://github.com/rdragos/awesome-mpc
- fresco-stat library: https://github.com/aicis/fresco-stat
- fresco-dp (Differential Privacy): https://github.com/aicis/fresco-dp
- SPDZ Protocol Paper: Damgard et al. (2012), CRYPTO
- TinyTables Protocol: Damgard et al. (2016), EUROCRYPT


Command Prompt
C:\Users\SAIKI\fresco\demos\sum>set JAVA_HOME=C:\Users\SAIKI\AppData\Local\Programs\Eclipse Adoptium\jdk-8.0.482.8-hotspot
C:\Users\SAIKI\fresco\demos\sum>set PATH=%JAVA_HOME%\bin;%PATH%
C:\Users\SAIKI\fresco\demos\sum>cd C:\Users\SAIKI\fresco\demos\sum
C:\Users\SAIKI\fresco\demos\sum>java -cp target\fresco-demo-sum.jar dk.alexandra.fresco.demo.TwoPartySum -i 1 -p 1:localhost:8001 -p 2:localhost:8002 -s dummyArithmetic
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Party 1 is holding secret value: 10
Party 1 is waiting for other party...
Secure computation complete!
Party 1 secret input : 10 (Party 2 never saw this)
Party 2 secret input : 15 (Party 1 never saw this)
Result (10 + 15) : 25
C:\Users\SAIKI\fresco\demos\sum>
0)
at java.net.ServerSocket.<init>(ServerSocket.java:252)
at java.net.ServerSocket.<init>(ServerSocket.java:143)
at javax.net.DefaultServerSocketFactory.createServerSocket(ServerSocketFactory.java:218)
at dk.alexandra.fresco.framework.network.socket.Connector.connectNetwork(Connector.java:80)
... 7 more
C:\Users\SAIKI\fresco\demos\sum>java -cp target\fresco-demo-sum.jar dk.alexandra.fresco.demo.TwoPartySum -i 2 -p 1:localhost:8001 -p 2:localhost:8002 -s dummyArithmetic
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Party 2 is holding secret value: 15
Party 2 is waiting for other party...
Secure computation complete!
Party 1 secret input : 10 (Party 2 never saw this)
Party 2 secret input : 15 (Party 1 never saw this)
Result (10 + 15) : 25
C:\Users\SAIKI\fresco\demos\sum>